Data Mining

Support Vector Machine

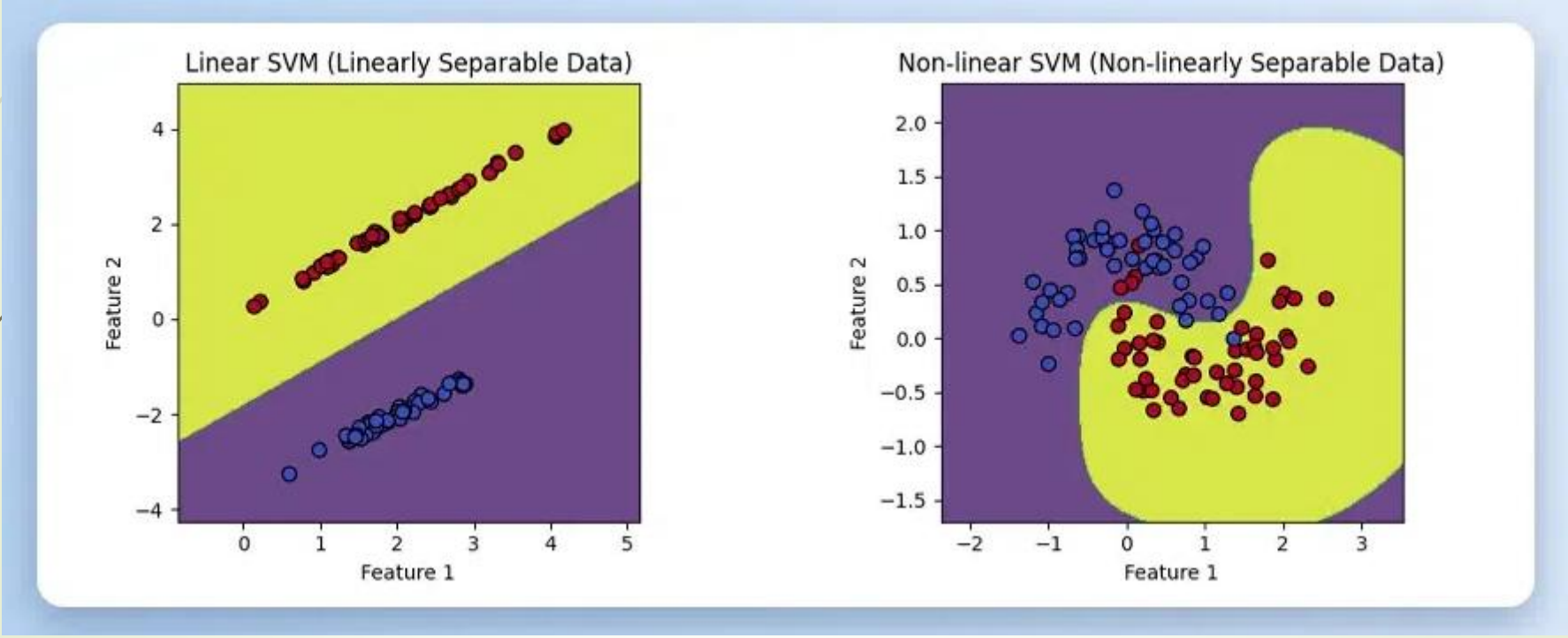
Introduction

- Supervised learning algorithm for classification & regression
- Goal: find the optimal separating hyperplane
- Maximizes the margin between different class boundaries
- Works well in high-dimensional spaces

Key Concepts

- Hyperplane: Decision boundary separating classes
- Margin: Distance between hyperplane & nearest data points
- Support Vectors: Critical data points that define the margin
- Optimal Hyperplane: Maximizes margin → better generalization

Linear Vs Non-Linear SVM



Linear Vs Non-Linear SVM

- Linear
  - Assumes data is linearly separable
  - Finds hyperplane that best separates two classes
  - Works well when features are linearly related
- Non-Linear
  - Real-world data is often not linearly separable
  - Kernel functions map data to higher dimensions
- Common kernels:
  - Linear Kernel – simple dot product
  - Polynomial Kernel – captures polynomial relations
  - Radial Basis Function (RBF) – handles complex boundaries

Advantages

- Effective in high-dimensional spaces
- Works well even with limited samples
- Memory efficient: only support vectors are needed
- Flexible via kernels

Disadvantages

- Computationally expensive for very large datasets
- Choice of kernel & parameters is critical
- Less interpretable compared to decision trees
- Struggles with overlapping class distributions

Data Mining

k-Nearest Neighbor Classifiers

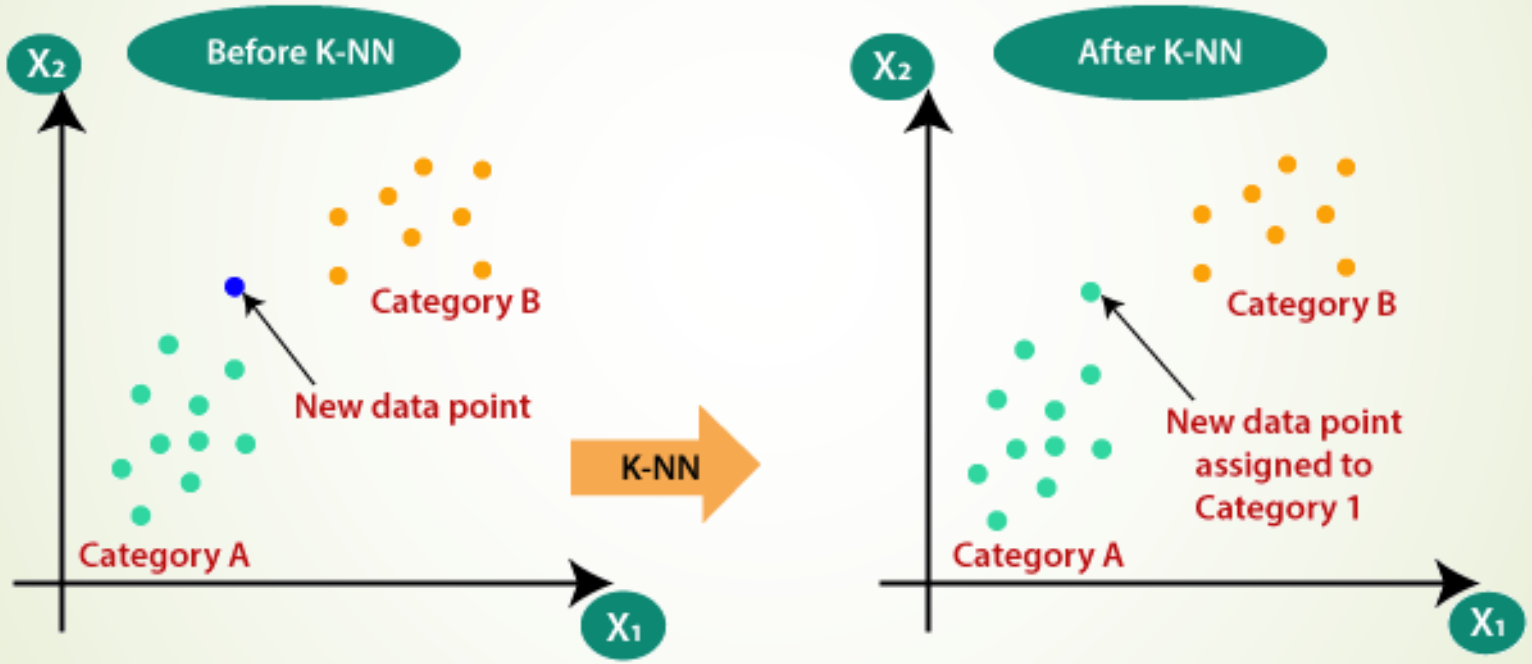
Introduction

- Instance-based (lazy) learning algorithm
- No explicit training model → classification done at query time
- Uses distance metrics to compare similarity
- Popular for simple yet effective classification tasks

Working Principle

- Store all training instances
- Given a new sample, compute distances to all training points
- Select the k nearest neighbors
- Predict class by majority vote among neighbors

Working Principle



Choice of k

- Small k → sensitive to noise, may overfit
- Large k → smoother decision boundary, may underfit
- Rule of thumb: odd k values for binary classification
- Common practice: tune k using cross-validation

Distance Metrics

- Euclidean Distance (most common)
- Manhattan Distance
- Minkowski Distance
- Cosine Similarity (for text/vector data)

Advantages

- Simple, intuitive, easy to implement
- No training phase → flexible
- Naturally handles multi-class problems
- Performs well with enough representative data

Disadvantages

- Computationally expensive at prediction time
- Requires storing the entire dataset
- Sensitive to irrelevant features & scaling
- Poor performance on high-dimensional data (“curse of dimensionality”)

Data Mining

Rule Based Classification

Rule Based Classification

- A technique that uses a set of IF-THEN rules to categorize data. It's highly valued for its simplicity and transparency (explainability).
- Rules are expressed as IF-THEN statements.
  - IF (Condition) THEN (Conclusion)
- Antecedent (IF Part): The condition part, typically a conjunction (AND) of attribute tests.
  - *Example:* (Age = 'Youth') ∧ (Student = 'Yes')
- Consequent (THEN Part): The predicted class label.
  - *Example:* THEN (Buys Computer = 'Yes')
- Complete Rule Example: IF (Age = 'Youth') ∧ (Student = 'Yes') THEN (Buys Computer = 'Yes')

Key Concepts for Rule Assessment

- Coverage: The fraction of training records that satisfy the rule's antecedent. A high coverage means the rule applies to a large portion of the data.
- Accuracy (Confidence): The fraction of records that satisfy the antecedent *and* are correctly classified by the consequent.
  - $\text{Accuracy} = \frac{\text{Number of correct classifications}}{\text{Number of records covered by the rule}}$
- Mutually Exclusive Rules: A set of rules where no two rules are triggered by the same record. Rule-based classifiers often use non-mutually exclusive rules, leading to potential conflicts.
- Exhaustive Rules: A set of rules that covers *all* possible combinations of attributes, meaning every record is covered by at least one rule.

Rule Induction (Learning Rules)

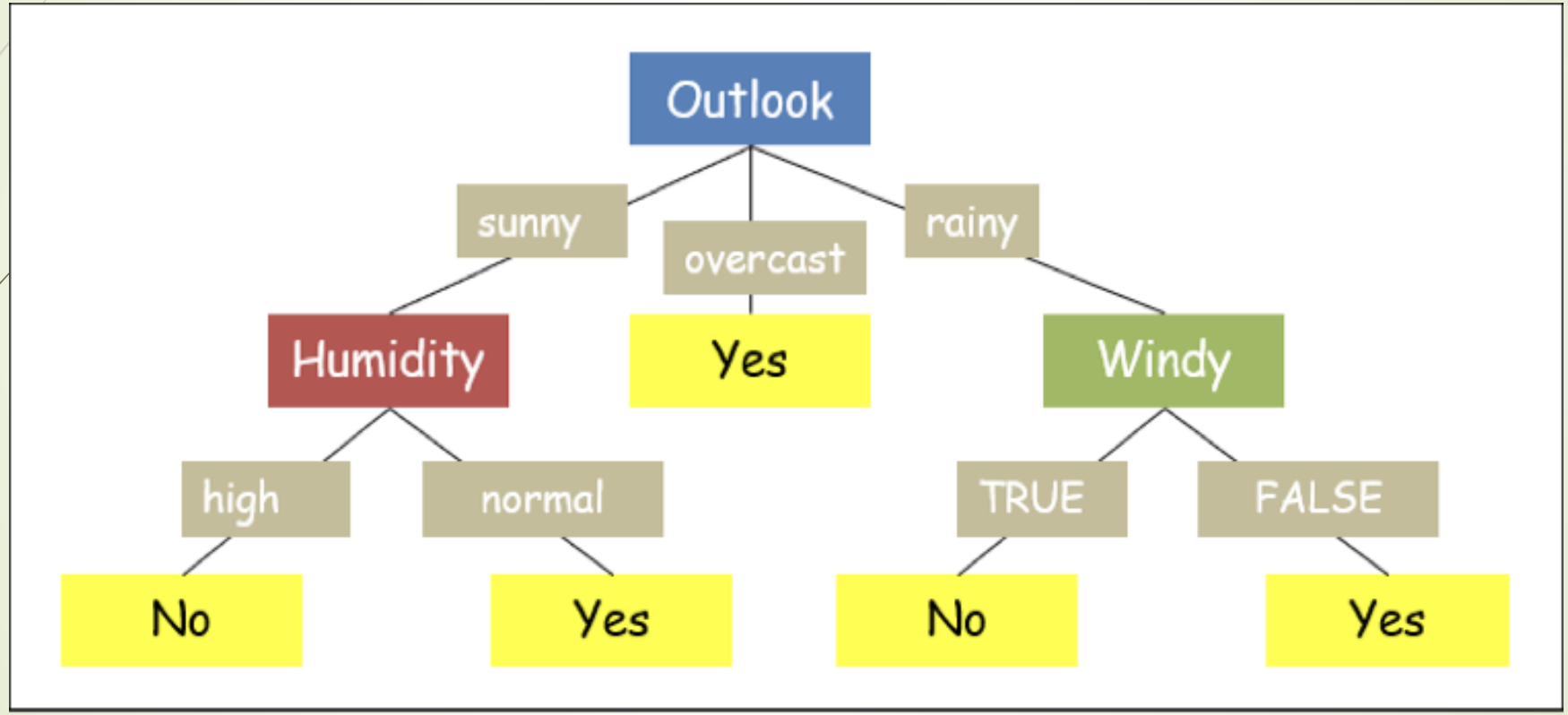
- Direct Method (Sequential Covering Algorithm):
  - Learns one rule at a time for a specific class.
- Indirect Method (From Decision Trees):

Decision Tree Example

Day	outlook	temp	humidity	windy	play
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

https://www.enjoyalgorithms.com/blog/decision-tree-algorithm-in-ml

Decision Tree Example



https://www.enjoyalgorithms.com/blog/decision-tree-algorithm-in-ml

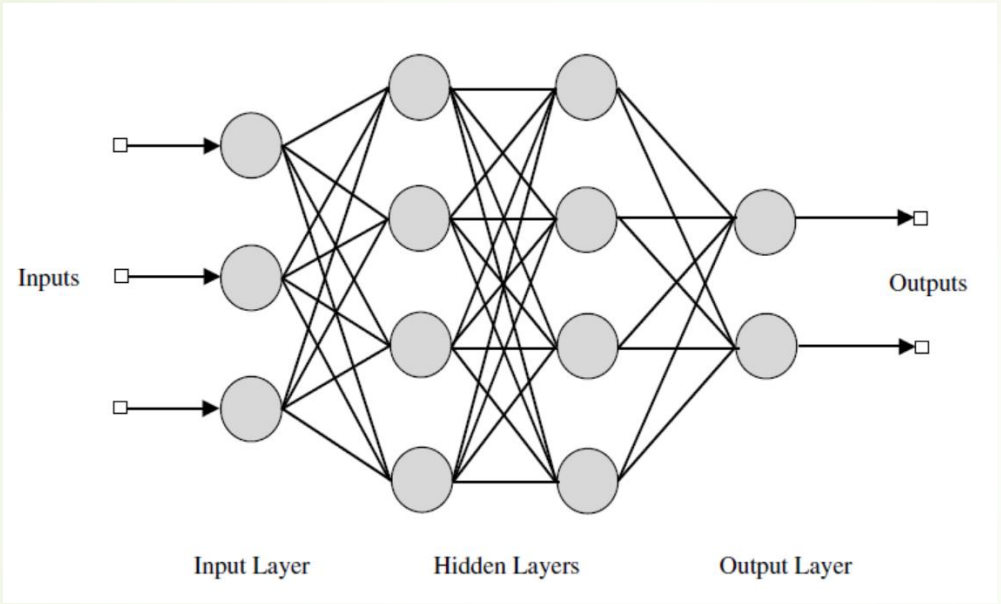
Data Mining

Multilayer Feed- Forward Neural Network

Multilayer Feed-Forward Neural Network

- A type of Artificial Neural Network (ANN)
- Composed of Input Layer, Hidden Layers, and Output Layer
- Fully connected: each node in one layer connects to all nodes in the next
- Processes data only in forward direction (no feedback loops)

Architecture



- Input Layer: Receives raw features
- Hidden Layers: Perform nonlinear transformations
- Output Layer: Produces predictions

How it works?

- Inputs go into the first layer (the input layer).
- They're transformed by weights and biases and passed through an activation function.
- This continues through one or more hidden layers.
- Finally, the output layer makes a prediction.

Input layer

- This is the entry point of the network.
- Each neuron here represents one feature from the dataset.
- The input layer does not perform any computation itself — it simply passes raw numbers forward.

Hidden layers

- These are where the real computation happens.
- Each neuron in a hidden layer does these three things:
- Weighted sum: Each input is multiplied by a weight and then all of them are summed.
- Bias: A constant bias is added to give flexibility in shifting the function.
- Activation function: A non-linear squashing function like ReLU, tanh, or sigmoid is applied. This step is what allows the network to learn complex, non-linear patterns.

Output layer

- The output layer produces the final prediction.
- Its design depends on the problem we are trying to solve:
  - Regression Problems: the output layer will contain one neuron with no activation (linear)
  - Binary Classification: output layer would be one neuron with a sigmoid activation
  - Multi-Label Classification: The output layer would have one neuron per class, with softmax activation.

Training with Backpropagation

- Forward Pass → compute predicted outputs
- Loss Function → measures prediction error (e.g., MSE, Cross-Entropy)
- Backward Pass → propagate error backwards to compute gradients
- Weight Update → use gradient descent or its variants (SGD, Adam)

Strengths

- Can model complex nonlinear relationships
- High accuracy in image, text, and speech domains
- Scalable to large problems with deep architectures
- General-purpose learner (works across domains)

Weaknesses

- Requires large labelled datasets
- Computationally expensive (needs GPUs/TPUs for deep nets)
- Training can be slow & difficult to tune
- Risk of overfitting without regularization